

B ve B+ Ağaçlarının Günümüzde Kullanım Alanları

Veri organizasyonu ödev-4

Veri Tabanlarında Kullanım

MySQL

InnoDB storage engine'i, birincil anahtar (primary key) indeksini **Clustered B+ Tree** olarak saklar. Yani veri satırları doğrudan yaprak düğümlerin içindedir. İkincil indeksler ise ayrı B+ Tree'ler olup yapraklarda birincil anahtarı gösterir. Bu yöntem özellikle BETWEEN, ORDER BY ve RANGE sorgularında büyük performans sağlar.

PostgreSQL

PostgreSQL, varsayılan indeks türü olarak B-Tree kullanır. **Heap tablosu** (veri) ile indeks ayrı tutulur; indeks yaprak düğümleri fiziksel satır adresini (TID) gösterir. Ayrıca PostgreSQL, B-Tree yanı sıra Hash, GiST, GIN ve BRIN indeks türlerini de destekler ancak genel kullanım B-Tree'dir.

MongoDB

MongoDB'nin WiredTiger motoru, koleksiyon ve indeks verilerini **B-Tree yapısında** saklar. Belge tabanlı (NoSQL) olmasına rağmen indeks mekanizması klasik B+ Tree mantığını izler. Bileşik indeksler (compound indexes) ve metin indeksleri de B-Tree tabanlıdır.

Neden veri tabanları b+ tree seçer ?

Veri tabanları çoğunlukla **disk üzerinde** çalışır. Bir disk bloğu (page) genellikle 4 KB – 16 KB büyüklüğündedir. B+ Tree'nin geniş dallı yapısı (fan-out), bir düğümde yüzlerce anahtar tutabildiğinden çok az disk erişimiyle milyonlarca kayda ulaşılabilir. Örneğin 100 milyon satırlık bir tabloda B+ Tree yüksekliği yalnızca 3–4 seviyedir yani yalnızca 3–4 disk okumasıyla herhangi bir kayıt bulunabilir.

Dosya Sistemlerinde Kullanım

Dosya sistemleri, disk üzerindeki dosya ve izin bilgilerini organize etmek için B-Tree ve B+ Tree'den yoğun biçimde yararlanır.

NTFS(WINDOWS)

Microsoft'un NTFS dosya sistemi, **Master File Table (MFT)** yapısını bir B+ Tree ile yönetir. Her dosya ve izin için meta veriler (isim, boyut, zaman damgası, izinler) bu ağaçta tutulur. Büyük izinlerde dosya arama işlemi $O(\log n)$ karmaşıklıkla gerçekleşir; milyonlarca dosya içeren bir klasörde bile arama anlık tamamlanır.

EXT4(LINUX)

Linux'un yaygın dosya sistemi ext4, **HTree (Hash Tree)** adlı bir varyasyon kullanır . Bu yapı esasen B-Tree prensiplerine dayanır. Büyük izinlerde dosya aramayı hızlandırmak için inode'ların hash değerleri B-Tree yapısında indekslenir.

APFS(APPLE) VE BTRFS

Apple'ın APFS dosya sistemi ve Linux'un yeni nesil Btrfs'i, metadata yönetimi için doğrudan **Copy-on-Write B-Tree** mimarisi kullanır. Bu yaklaşım hem snapshots (anlık görüntü) hem de veri bütünlüğü açısından büyük avantajlar sağlar.

Dosya Sistemleri İçin Kritik Avantaj

Dosya sistemleri **güç kesintisi** ve **ani kapanma** gibi durumlarda tutarlı kalmalıdır. B+ Tree'nin dengeli yapısı, yarım kalan işlemlerin kurtarılmasını (recovery) mümkün kılar. WAL (Write-Ahead Log) ile birleştirildiğinde ACID uyumlu işlemler gerçekleştirilebilir.

Arama Motorları ve Büyük Veri

ELASTICSEARCH VE APACHE LUCENE

Elasticsearch'ün altında yatan Lucene kütüphanesi, terim (term) sözlükleri için **B-Tree benzeri yapılar** kullanır. Bir arama terimi girildiğinde Lucene'in term dictionary'si bu terime karşılık gelen doküman listesine (posting list) $O(\log n)$ sürede ulaşır. Milyarlarca belge indekslenmiş ortamlarda bu yapı sayesinde milisaniyeler içinde arama sonuçları üretilir.

APACHE HBASE VE CASSANDRA

Dağıtık NoSQL veri tabanları olan HBase ve Cassandra, **LSM-Tree (Log-Structured Merge Tree)** kullanır — bu yapı B-Tree'den türetilmiştir. Yazma işlemleri önce bellekte tutulur, ardından disk üzerindeki B+ Tree benzeri SSTables'a birleştirilir (compaction). Bu sayede milyonlarca yazma işlemi saniyede gerçekleştirilebilir.

Büyük veri için neden kritik?

Petabaytlık veri kümelerinde bile B+ Tree'nin **$O(\log n)$ garanti performansı** değişmez. 1 milyar kayıt için B+ Tree yüksekliği yaklaşık 30 adımdır. Doğrusal ($O(n)$) bir yapı ise 1 milyar adım gerektirir bu fark saniyeler ile saatler arasındaki farktır.

Kütüphane Sistemi Örneği

B-Tree ve B+ Tree yapılarını soyut matematiksel kavramlar olarak değil, günlük hayattan somut bir örnekle açıklamak çok daha kalıcı bir anlayış sağlar. Ve aranan veriyi daha hızlı bulunmasını sağlar. Bu amaçla büyük bir kütüphanede kitap arama sistemini kullanabiliriz.

Kütüphane Yapısı

Fiziksel Düzenleme

- ⇒ Kütüphane ⇨ Veritabanı
- ⇒ Raf ⇨ Disk Bloğu
- ⇒ Kitap ⇨ Veri Kaydı
- ⇒ Katalog kartı ⇨ İndeks Anahtarı
- ⇒ Bölüm tabelası ⇨ İç Düğüm
- ⇒ Katalog çekmeceleri ⇨ Yaprak Düğüm

Arama süreci nasıl çalışır?

Kütüphaneye giriyorsunuz. "Orhan Pamuk" adlı bir yazarın kitabını arıyorsunuz.

1. Ana girişte büyük tabela: "*A–K → Sol kanat, L–Z → Sağ kanat*" — Bu B-Tree'nin kök düğümüdür.

2. Sol kanatta alt tabela: "*O harfi → 3. koridor*"

3. koridorda katalog kartını buluyorsunuz → Kitap tam yerinde!
Toplam 3 adım, 100.000 kitap arasından.

B+ Tree Farkı: Komşu Raflar Zincirleme Bağlı

B+ Tree'nin yaprak düğümlerinin birbirine bağlı olması şu anlama gelir: "Orhan Pamuk'un **tüm kitaplarını** listele" dediğinizde, ilk kitabı bulduktan sonra katalog kartları birbirine zincirleme bağlı olduğundan bir sonraki kartı aramak için en başa dönmenize **gerek yok**. Doğrudan bir sonraki karta geçersiniz.

Modern Sistemlerde Neden Tercih Edilir?

1-Disk I/O Optimizasyonu

Modern depolama sistemlerinde en pahalı işlem disk okumadır . B+ Tree'nin yüksek fan-out değeri sayesinde disk erişim sayısı minimuma iner. Milyar kayıtlı bir tabloda 4–5 disk bloğu okumak yeterlidir.

2-Garantili Performans

Hash indeksleri eşitlik sorgularında çok hızlıdır ancak aralık sorgularını desteklemez ve en kötü durumda $O(n)$ 'e düşebilir. B+ Tree ise arama, ekleme, silme için her zaman $O(\log n)$ garantisi sunar.

3-Dinamik Veri Setleri

Veriler sürekli eklenir ve silinir. B+ Tree, her işlem sonrasında kendini otomatik olarak dengeler (rebalancing). Ağaç hiçbir zaman eğri büğrü bir hâl almaz; bu da tutarlı performans anlamına gelir

4-Eş Zamanlı Erişim (Concurrency)

Çok kullanıcıli sistemlerde farklı kullanıcılar aynı anda farklı düğümlere erişebilir. B+ Tree'nin ayrık düğüm yapısı, kilitleme mekanizmalarını kolaylaştırır; böylece yüksek eş zamanlılık sağlanır.

5-Cache Dostu Yapı

B+ Tree'nin iç düğümleri sık erişilen kısımlardır ve işletim sistemi cache'inde tutulur. Bir sorgunun kök ve ara düğümleri zaten bellekte olduğundan yalnızca son yaprak düğüm için disk erişimi gerekir bu da pratik performansı teorik sınırın çok üzerine taşır.

Bu veri yapıları neden önemlidir?

B-Tree ve B+ Tree yapıları; milyonlarca veri içinde saniyeler içinde arama yaparak aranan veriyi kısa süre içinde bulmamızı sağlar eğer olmasaydı tüm veritabanını baştan taramak zorunda kalırdık bu da kullanıcıyı baya bekletirdi ve sunucuları kilitlerdi.

Büyük veri çağında neden ihtiyaç duyulmaktadır?

günümüzde üretilen veri miktarı her yıl katlanarak artmaktadı B+ Tree nin özelliği veri miktarı büyüdükçe performansın logaritmik biçimde azalmasıdır.böyle bir yapı veri dünyasının vazgeçilmezidir.

Sizce gelecekte de kullanılmaya devam edilecek midir ?

Yazma odaklı işlerde yerini LSM-Tree'ye bıraksa da arkasındaki o devasa güvenilirlik ve temel mantık yüzünden bence B+ Tree bir 20-30 yıl daha sistemlerin merkezinden silinmez.